

BCA

III
SEMESTER

DATABASE MANAGEMENT SYSTEM

BCA

III SEMESTER



JAI NARAYAN VYAS
UNIVERSITY, JODHPUR

ACCORDING TO
LATEST
SYLLABUS

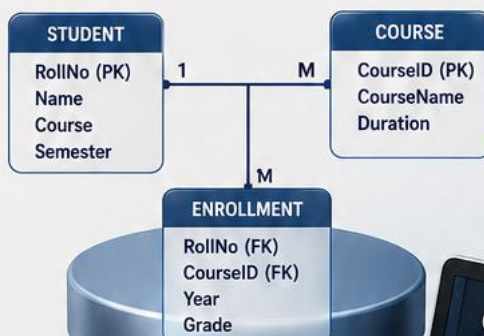


DATABASE MANAGEMENT SYSTEM

A COMPLETE TEXTBOOK FOR BCA STUDENTS

COVERS

-  Introduction to DBMS
-  Data Models & ER Model
-  Structured Query Language (SQL)
-  Relational Model & Relational Algebra
-  Normalization
-  Transaction Management
-  Concurrency Control
-  File Organization & Indexing
-  Backup & Recovery
-  Practical Programs & Case Studies



EASY
EXPLANATION



CONCEPTS WITH
EXAMPLES



EXAM
FOCUSED



DIAGRAMS & TABLES
FOR BETTER
UNDERSTANDING

• LEARN TODAY, LEAD TOMORROW •

UNIT FIRST

Topic 01 Database Management System

डेटाबेस प्रबंधन प्रणाली (DBMS) एक सॉफ्टवेयर है जो डेटा को व्यवस्थित, सुरक्षित और कुशलतापूर्वक प्रबंधित करने में मदद करता है। यह उपयोगकर्ता और डेटाबेस के बीच एक इंटरफेस की तरह काम करता है, जिससे डेटा को आसानी से एक्सेस, अपडेट और संरक्षित किया जा सकता है।

परिभाषा (Definition)

डेटाबेस प्रबंधन प्रणाली (DBMS) एक ऐसा सॉफ्टवेयर है जो डेटा को संग्रहित करने, संशोधित करने, पुनः प्राप्त करने और सुरक्षित रखने के लिए उपयोग किया जाता है। यह उपयोगकर्ता और डेटाबेस के बीच एक मध्यस्थ के रूप में कार्य करता है।

उदाहरण: जब कोई छात्र का नाम, रोल नंबर और कोर्स की जानकारी एक टेबल में संग्रहित की जाती है, तो DBMS उस टेबल को बनाने, उसमें डेटा जोड़ने और उसे एक्सेस करने की सुविधा देता है।

DBMS की आवश्यकता क्यों है?

पारंपरिक फ़ाइल सिस्टम में कई समस्याएँ होती थीं:

- डेटा की दोहराव (Redundancy)
- डेटा की असंगति (Inconsistency)
- सुरक्षा की कमी
- डेटा को खोजने और अपडेट करने में कठिनाई

DBMS इन समस्याओं को हल करता है:

- डेटा को केंद्रीकृत रूप से संगठित करता है
- एक्सेस कंट्रोल और सुरक्षा प्रदान करता है
- एक ही डेटा को कई उपयोगकर्ता एक साथ उपयोग कर सकते हैं

DBMS के मुख्य घटक (Components of DBMS)

घटक	कार्य
डेटाबेस	डेटा का संग्रह
DBMS सॉफ्टवेयर	डेटा को प्रबंधित करने वाला सॉफ्टवेयर
हार्डवेयर	सर्वर, स्टोरेज आदि
उपयोगकर्ता	DBA, डेवलपर, एंड यूज़र

DBMS के प्रकार

1. **Relational DBMS (RDBMS)** – डेटा को टेबल्स में संगठित करता है
उदाहरण: MySQL, Oracle, PostgreSQL
2. **NoSQL DBMS** – डेटा को key-value pairs, documents आदि में संगठित करता है
उदाहरण: MongoDB, Cassandra

DBMS के लाभ (Advantages)

- डेटा सुरक्षा: पासवर्ड और एक्सेस कंट्रोल
 - डेटा स्वतंत्रता: एप्लिकेशन को बदले बिना डेटा संरचना को बदलना संभव
 - डेटा एकता और सुसंगति: एक ही डेटा का एक संस्करण
 - क्वेरी भाषा: SQL जैसी भाषाओं से डेटा को आसानी से एक्सेस करना
-

DBMS का आरेख (Diagram)

[User/Application]



[DBMS Engine]



[Database]

DBMS का उदाहरण (SQL Snippet)

```
CREATE TABLE Students (  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Course VARCHAR(50)  
);
```

Topic 02 Data Abstraction

परिभाषा (Definition)

डेटा एब्स्ट्रैक्शन का अर्थ है डेटा को अलग-अलग स्तरों पर प्रस्तुत करना ताकि उपयोगकर्ता केवल वही जानकारी देख सके जो उसके लिए आवश्यक है।

DBMS में डेटा एब्स्ट्रैक्शन का उद्देश्य है जटिलता को छुपाना और सरल इंटरफ़ेस प्रदान करना।

डेटा एब्स्ट्रैक्शन के स्तर (Levels of Data Abstraction)

DBMS में डेटा को तीन स्तरों पर देखा जाता है:

1. आंतरिक स्तर (Internal Level)

- यह सबसे निचला स्तर है।
- इसमें डेटा वास्तव में कैसे स्टोर होता है (फाइल्स, इंडेक्स, ब्लॉक्स) यह बताया जाता है।
- उपयोगकर्ता को यह स्तर दिखाई नहीं देता।
- उदाहरण: डेटा डिस्क पर किस फॉर्मेट में सेव हुआ है।

2. अवधारणात्मक स्तर (Conceptual Level)

- यह डेटाबेस का तार्किक दृश्य (Logical View) है।
- इसमें बताया जाता है कि कौन-कौन सी टेबल्स हैं, उनके बीच संबंध (Relationships) क्या हैं।
- उदाहरण: "Students" और "Courses" टेबल्स तथा उनके बीच का संबंध।

3. बाह्य स्तर (External Level)

- यह उपयोगकर्ता का दृष्टिकोण (User View) है।
- हर उपयोगकर्ता को केवल वही डेटा दिखाया जाता है जो उसके लिए आवश्यक है।
- उदाहरण: एक छात्र केवल अपनी जानकारी देख सकता है, जबकि शिक्षक सभी छात्रों की जानकारी देख सकता है।

आरेख (Diagram)

[External Level] → उपयोगकर्ता का दृश्य
[Conceptual Level] → डेटाबेस का तार्किक दृश्य
[Internal Level] → डेटा का भौतिक संग्रह

उदाहरण (Example)

मान लीजिए हमारे पास एक **Students** टेबल है:

```
CREATE TABLE Students (  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Course VARCHAR(50),  
    Marks INT  
);
```

- **Internal Level:** डेटा डिस्क पर किस फॉर्मेट में सेव हुआ है।
- **Conceptual Level:** Students टेबल में RollNo, Name, Course, Marks कॉलम हैं।
- **External Level:**
 - छात्र को केवल RollNo, Name, Marks दिखाया जाए।
 - शिक्षक को RollNo, Name, Course, Marks दिखाया जाए।

डेटा एब्स्ट्रैक्शन के लाभ (Advantages)

- उपयोगकर्ता को केवल आवश्यक जानकारी दिखती है।
- जटिलता कम होती है।
- सुरक्षा बढ़ती है (Sensitive डेटा छुपाया जा सकता है)।
- डेटा स्वतंत्रता (Data Independence) प्राप्त होती है।

Topic 03 Data Models

परिभाषा (Definition)

डेटा मॉडल वह तरीका है जिससे डेटाबेस में डेटा को संगठित और प्रस्तुत किया जाता है। यह बताता है कि डेटा कैसे संग्रहित होगा, उसके बीच संबंध कैसे होंगे और उपयोगकर्ता उसे कैसे एक्सेस करेगा।

डेटा मॉडल के प्रकार (Types of Data Models)

1. हायरैरिकल मॉडल (Hierarchical Model)

- डेटा को पेड़ (Tree) संरचना में संगठित किया जाता है।

- प्रत्येक पेरेंट नोड के कई चाइल्ड नोड हो सकते हैं।
- उदाहरण: एक "कंपनी" के अंतर्गत कई "विभाग", और प्रत्येक विभाग के अंतर्गत कई "कर्मचारी"।

आरेख

Company

└─ Department

└─ Employee

2. नेटवर्क मॉडल (Network Model)

- डेटा को ग्राफ संरचना में संगठित किया जाता है।
- एक रिकॉर्ड कई रिकॉर्ड्स से जुड़ा हो सकता है।
- उदाहरण: छात्र कई कोर्स कर सकता है और एक कोर्स में कई छात्र हो सकते हैं।

आरेख

Student ↔ Course

3. रिलेशनल मॉडल (Relational Model)

- सबसे लोकप्रिय मॉडल।
- डेटा को टेबल्स (Relations) में संगठित किया जाता है।
- प्रत्येक टेबल में पंक्तियाँ (Rows) और स्तंभ (Columns) होते हैं।
- उदाहरण: Students टेबल जिसमें RollNo, Name, Course कॉलम हों।

SQL उदाहरण

```
CREATE TABLE Students (
  RollNo INT PRIMARY KEY,
  Name VARCHAR(50),
  Course VARCHAR(50)
);
```

4. एंटिटी-रिलेशनशिप मॉडल (Entity-Relationship Model)

- डेटा को Entities (जैसे Student, Course) और उनके संबंधों (Relationships) के रूप में दर्शाया जाता है।
- ER Diagram का उपयोग होता है।

आरेख

Student ---enrolled_in---> Course

5. ऑब्जेक्ट-ओरिएंटेड मॉडल (Object-Oriented Model)

- डेटा को ऑब्जेक्ट्स के रूप में संगठित किया जाता है।
- इसमें डेटा और उससे संबंधित मेथड्स दोनों शामिल होते हैं।
- उदाहरण: "Student" ऑब्जेक्ट जिसमें Name, RollNo और एक मेथड calculateGrade() हो।

डेटा मॉडल के लाभ (Advantages)

- डेटा को व्यवस्थित रूप से प्रस्तुत करता है।
- उपयोगकर्ता को डेटा समझने और उपयोग करने में आसानी होती है।
- विभिन्न एप्लिकेशन के लिए उपयुक्त संरचना प्रदान करता है।
- डेटा स्वतंत्रता और सुरक्षा सुनिश्चित करता है।

Topic 04 Data Independence

परिभाषा (Definition)

डेटा स्वतंत्रता का अर्थ है डेटाबेस की संरचना (Structure) में बदलाव करने पर भी एप्लिकेशन प्रोग्राम पर कोई प्रभाव न पड़े।

DBMS में यह सुविधा होती है कि हम डेटा को अलग-अलग स्तरों पर बदल सकते हैं, लेकिन उपयोगकर्ता और एप्लिकेशन को वही दृश्य मिलता है जो उनके लिए आवश्यक है।

प्रकार (Types of Data Independence)

1. भौतिक डेटा स्वतंत्रता (Physical Data Independence)

- इसमें डेटा के स्टोरेज फॉर्मेट या फाइल संरचना में बदलाव करने पर भी एप्लिकेशन प्रभावित नहीं होता।
- उदाहरण: यदि डेटा को डिस्क पर अलग-अलग ब्लॉक्स में स्टोर किया जाए या इंडेक्सिंग बदली जाए, तो भी उपयोगकर्ता को वही टेबल दिखाई देगी।

2. तार्किक डेटा स्वतंत्रता (Logical Data Independence)

- इसमें डेटाबेस की तार्किक संरचना (जैसे टेबल्स, कॉलम्स, रिलेशनशिप) में बदलाव करने पर भी एप्लिकेशन प्रभावित नहीं होता।
- उदाहरण: यदि Students टेबल में नया कॉलम "Email" जोड़ दिया जाए, तो भी पुराने एप्लिकेशन जो केवल RollNo और Name का उपयोग करते हैं, वे बिना किसी बदलाव के काम करेंगे।

आरेख (Diagram)

[External Level] → उपयोगकर्ता का दृश्य

[Conceptual Level] → तार्किक संरचना

[Internal Level] → भौतिक संग्रह

डेटा स्वतंत्रता = एक स्तर पर बदलाव करने पर दूसरे स्तर पर असर न होना

उदाहरण (SQL Snippet)

-- Students टेबल में नया कॉलम जोड़ना

```
ALTER TABLE Students ADD Email VARCHAR(100);
```

-- पुराने एप्लिकेशन अभी भी RollNo और Name से काम कर सकते हैं

```
SELECT RollNo, Name FROM Students;
```

यहाँ **Logical Data Independence** दिखता है क्योंकि नया कॉलम जोड़ने पर भी पुराने क्वेरी प्रभावित नहीं हुए।

लाभ (Advantages of Data Independence)

- एप्लिकेशन और डेटाबेस को अलग-अलग विकसित और संशोधित किया जा सकता है।
- सिस्टम की लचीलापन (Flexibility) बढ़ता है।
- रखरखाव (Maintenance) आसान होता है।

Topic 04 Data Definition Language

परिभाषा (Definition)

डेटा डेफिनिशन लैंग्वेज (DDL) SQL का वह भाग है जिसका उपयोग डेटाबेस की संरचना (Structure) को परिभाषित करने और संशोधित करने के लिए किया जाता है।

DDL कमांड्स का उपयोग टेबल्स, स्कीमा, इंडेक्स, व्यूज़ आदि बनाने, बदलने और हटाने के लिए किया जाता है।

DDL की मुख्य कमांड्स (Major Commands of DDL)

1. CREATE

- नई टेबल, डेटाबेस, इंडेक्स या व्यू बनाने के लिए।
- उदाहरण:
- `CREATE TABLE Students (`
- `RollNo INT PRIMARY KEY,`
- `Name VARCHAR(50),`
- `Course VARCHAR(50)`
- `);`

2. ALTER

- मौजूदा टेबल की संरचना बदलने के लिए।
- उदाहरण:
- `ALTER TABLE Students ADD Email VARCHAR(100);`

3. DROP

- टेबल, डेटाबेस या अन्य ऑब्जेक्ट को हटाने के लिए।
- उदाहरण:
- `DROP TABLE Students;`

4. TRUNCATE

- टेबल की सभी पंक्तियाँ हटाने के लिए, लेकिन संरचना बनी रहती है।
- उदाहरण:
- `TRUNCATE TABLE Students;`

5. COMMENT

- डेटाबेस ऑब्जेक्ट्स पर टिप्पणी (Documentation) जोड़ने के लिए।
- उदाहरण:
- `COMMENT ON TABLE Students IS 'यह टेबल छात्रों की जानकारी रखती है';`

DDL की विशेषताएँ (Features of DDL)

- डेटाबेस संरचना को परिभाषित करता है

- डेटा पर नहीं, बल्कि स्कीमा पर कार्य करता है
- ऑटो-कमिट होता है (यानी DDL कमांड्स चलने के बाद बदलाव तुरंत सेव हो जाते हैं)
- डेटा स्वतंत्रता को सपोर्ट करता है

आरेख (Diagram)

User → SQL (DDL Commands) → DBMS → Database Schema

उदाहरण (Example with Flow)

1. CREATE TABLE

```
2. CREATE TABLE Courses (
3.   CourseID INT PRIMARY KEY,
4.   CourseName VARCHAR(50)
5. );
```

6. ALTER TABLE

```
7. ALTER TABLE Courses ADD Duration INT;
```

8. DROP TABLE

```
9. DROP TABLE Courses;
```

Topic 05 Data Manipulation Language

परिभाषा (Definition)

डेटा मैनिपुलेशन लैंग्वेज (DML) SQL का वह भाग है जिसका उपयोग डेटाबेस में मौजूद डेटा को जोड़ने (Insert), बदलने (Update), हटाने (Delete) और प्राप्त करने (Select) के लिए किया जाता है। DML कमांड्स सीधे टेबल्स के डेटा पर कार्य करते हैं, न कि उनकी संरचना पर।

DML की मुख्य कमांड्स (Major Commands of DML)

1. INSERT

- टेबल में नया डेटा जोड़ने के लिए।
- उदाहरण:
- INSERT INTO Students (RollNo, Name, Course, Marks)
- VALUES (101, 'Amit', 'DBMS', 85);

2. UPDATE

- मौजूदा डेटा को बदलने के लिए।
- उदाहरण:
- UPDATE Students
- SET Marks = 90
- WHERE RollNo = 101;

3. DELETE

- टेबल से डेटा हटाने के लिए।
- उदाहरण:

- DELETE FROM Students
- WHERE RollNo = 101;

4. SELECT

- टेबल से डेटा प्राप्त करने के लिए।
- उदाहरण:
- SELECT Name, Marks
- FROM Students
- WHERE Course = 'DBMS';

DML की विशेषताएँ (Features of DML)

- डेटा पर कार्य करता है, संरचना पर नहीं।
- **TCL (Transaction Control Language)** के साथ उपयोग होता है (जैसे COMMIT, ROLLBACK)।
- दो प्रकार:
 - **Procedural DML:** उपयोगकर्ता बताता है कि डेटा कैसे प्राप्त करना है।
 - **Non-Procedural DML:** उपयोगकर्ता केवल बताता है कि कौन सा डेटा चाहिए, DBMS तय करता है कि कैसे निकाला जाए।

आरेख (Diagram)

User → SQL (DML Commands) → DBMS → Database Data

उदाहरण (Flow Example)

1. डेटा जोड़ना (INSERT)
2. INSERT INTO Students VALUES (102, 'Ravi', 'OS', 78);
3. डेटा बदलना (UPDATE)
4. UPDATE Students SET Course = 'Networking' WHERE RollNo = 102;
5. डेटा हटाना (DELETE)
6. DELETE FROM Students WHERE RollNo = 102;
7. डेटा प्राप्त करना (SELECT)
8. SELECT * FROM Students;

लाभ (Advantages of DML)

- उपयोगकर्ता को डेटा पर सीधे नियंत्रण मिलता है।
- डेटा को आसानी से जोड़ना, बदलना और हटाना संभव।
- क्वेरी भाषा (SQL) के माध्यम से डेटा को तेज़ी से प्राप्त किया जा सकता है।
- ट्रांज़ैक्शन मैनेजमेंट के साथ सुरक्षित डेटा संचालन।

Topic 06 Data Query Language

परिभाषा (Definition)

डेटा क्वेरी लैंग्वेज (DQL) SQL का वह भाग है जिसका उपयोग डेटाबेस से डेटा **प्राप्त (Retrieve)** करने के लिए किया जाता है।

DQL का सबसे महत्वपूर्ण कमांड है **SELECT**, जिसके माध्यम से उपयोगकर्ता अपनी आवश्यकता अनुसार डेटा निकाल सकता है।

DQL की मुख्य कमांड (Major Command)

1. SELECT

- टेबल से डेटा प्राप्त करने के लिए।
- सिंटैक्स:
 - `SELECT column1, column2, ...`
 - `FROM table_name`
 - `WHERE condition;`

SELECT कमांड के विभिन्न उपयोग (Different Uses of SELECT)

1. सभी कॉलम प्राप्त करना

2. `SELECT * FROM Students;`

- ☐ यह Students टेबल की सभी पंक्तियाँ और कॉलम दिखाएगा।

3. विशिष्ट कॉलम प्राप्त करना

4. `SELECT Name, Course FROM Students;`

- ☐ केवल Name और Course कॉलम दिखाएगा।

5. शर्त के साथ डेटा प्राप्त करना (WHERE Clause)

6. `SELECT Name, Marks`

7. `FROM Students`

8. `WHERE Marks > 80;`

- ☐ केवल वे छात्र जिनके अंक 80 से अधिक हैं।

9. डेटा को क्रमबद्ध करना (ORDER BY Clause)

10. `SELECT Name, Marks`

11. `FROM Students`

12. `ORDER BY Marks DESC;`

- ☐ छात्रों को उनके अंकों के आधार पर घटते क्रम में दिखाएगा।

13. समूह बनाना (GROUP BY Clause)

14. `SELECT Course, AVG(Marks)`

15. `FROM Students`

16. `GROUP BY Course;`

- ☐ प्रत्येक कोर्स के औसत अंक दिखाएगा।

17. एग्रीगेट फंक्शन्स (Aggregate Functions)

- `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`

18. SELECT COUNT(*) FROM Students;

□ कुल छात्रों की संख्या बताएगा।

DQL की विशेषताएँ (Features of DQL)

- केवल डेटा प्राप्त करने के लिए उपयोग होता है।
- डेटा को बदला नहीं जाता, केवल देखा जाता है।
- विभिन्न शर्तों और फंक्शन्स के साथ डेटा को विश्लेषित किया जा सकता है।
- रिपोर्टिंग और डेटा एनालिसिस के लिए सबसे महत्वपूर्ण।

आरेख (Diagram)

User → SQL (DQL: SELECT) → DBMS → Database → Result Set

उदाहरण (Flow Example)

-- सभी छात्रों की जानकारी

```
SELECT * FROM Students;
```

-- केवल DBMS कोर्स वाले छात्र

```
SELECT Name, Marks FROM Students WHERE Course = 'DBMS';
```

-- सबसे अधिक अंक प्राप्त करने वाला छात्र

```
SELECT Name, MAX(Marks) FROM Students;
```

Topic 07 Database Manager

परिभाषा (Definition)

डेटाबेस मैनेजर DBMS का वह घटक है जो डेटाबेस के सभी कार्यों को नियंत्रित और प्रबंधित करता है। यह उपयोगकर्ता के अनुरोधों को संभालता है, डेटा को सुरक्षित रखता है और डेटाबेस की दक्षता (Efficiency) सुनिश्चित करता है।

डेटाबेस मैनेजर की भूमिकाएँ (Roles of Database Manager)

1. डेटा संग्रहण और पुनः प्राप्ति (Storage & Retrieval)

- उपयोगकर्ता द्वारा दिए गए क्वेरी को प्रोसेस करता है।
- डेटा को डिस्क से निकालकर उपयोगकर्ता तक पहुँचाता है।

2. डेटा सुरक्षा (Data Security)

- केवल अधिकृत उपयोगकर्ताओं को ही डेटा तक पहुँचने देता है।
- पासवर्ड और एक्सेस कंट्रोल लागू करता है।

3. डेटा अखंडता (Data Integrity)

- सुनिश्चित करता है कि डेटा सही और सुसंगत (Consistent) रहे।
- Constraints (जैसे Primary Key, Foreign Key) लागू करता है।

4. Concurrency Control (समानांतर नियंत्रण)

- कई उपयोगकर्ताओं द्वारा एक साथ डेटा एक्सेस करने पर टकराव (Conflicts) रोकता है।
- उदाहरण: दो छात्र एक ही समय पर Marks अपडेट कर रहे हों, तो सही परिणाम सुनिश्चित करता है।

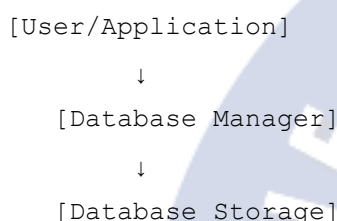
5. बैकअप और रिकवरी (Backup & Recovery)

- सिस्टम फेल होने पर डेटा को सुरक्षित रखता है।
- बैकअप से डेटा को पुनः प्राप्त करता है।

6. प्रदर्शन अनुकूलन (Performance Optimization)

- क्वेरी को तेज़ी से चलाने के लिए इंडेक्सिंग और कैशिंग का उपयोग करता है।
- डेटाबेस की गति और दक्षता बढ़ाता है।

आरेख (Diagram)



उदाहरण (Example)

मान लीजिए छात्र Marks अपडेट करना चाहता है:

```
UPDATE Students SET Marks = 95 WHERE RollNo = 101;
```

- **User** क्वेरी भेजता है।
- **Database Manager** क्वेरी को प्रोसेस करता है।
- यह सुनिश्चित करता है कि RollNo = 101 मौजूद है और Marks अपडेट हो।
- डेटा सुरक्षित रूप से डिस्क पर लिखा जाता है।

लाभ (Advantages of Database Manager)

- डेटा को सुरक्षित और सुसंगत रखता है।
- कई उपयोगकर्ताओं को एक साथ काम करने की सुविधा देता है।
- सिस्टम फेल होने पर भी डेटा सुरक्षित रहता है।
- डेटाबेस का प्रदर्शन बेहतर होता है।

Topic 08 Database Administrator

परिभाषा (Definition)

डेटाबेस एडमिनिस्ट्रेटर (DBA) वह व्यक्ति होता है जो डेटाबेस सिस्टम का प्रबंधन, रखरखाव और सुरक्षा सुनिश्चित करता है। DBA यह देखता है कि डेटाबेस सही ढंग से काम करे, डेटा सुरक्षित रहे और उपयोगकर्ताओं को आवश्यकतानुसार एक्सेस मिले।

DBA की मुख्य जिम्मेदारियाँ (Responsibilities of DBA)

- 1. डेटाबेस डिज़ाइन (Database Design)**
 - डेटाबेस की संरचना तैयार करना।
 - टेबल्स, रिलेशनशिप और constraints को परिभाषित करना।
- 2. डेटा सुरक्षा (Data Security)**
 - केवल अधिकृत उपयोगकर्ताओं को ही डेटा तक पहुँचने देना।
 - पासवर्ड, यूज़र रोल और एक्सेस कंट्रोल लागू करना।
- 3. डेटा अखंडता (Data Integrity)**
 - सुनिश्चित करना कि डेटा सही और सुसंगत रहे।
 - Primary Key, Foreign Key, Unique constraints का पालन कराना।
- 4. बैकअप और रिकवरी (Backup & Recovery)**
 - नियमित रूप से डेटाबेस का बैकअप लेना।
 - सिस्टम फेल होने पर डेटा को पुनः प्राप्त करना।
- 5. प्रदर्शन अनुकूलन (Performance Optimization)**
 - क्वेरी को तेज़ी से चलाने के लिए इंडेक्सिंग और ट्यूनिंग करना।
 - डेटाबेस की गति और दक्षता बढ़ाना।
- 6. यूज़र मैनेजमेंट (User Management)**
 - नए उपयोगकर्ताओं को जोड़ना।
 - उनके अधिकार (Privileges) तय करना।
- 7. Concurrency Control (समानांतर नियंत्रण)**
 - कई उपयोगकर्ताओं द्वारा एक साथ डेटा एक्सेस करने पर टकराव रोकना।

आरेख (Diagram)

[Users]



[DBA] → Database Design, Security, Backup, Performance



[Database System]

उदाहरण (Example)

मान लीजिए DBA को छात्रों के डेटाबेस में सुरक्षा लागू करनी है:

-- नया उपयोगकर्ता बनाना

```
CREATE USER student_user IDENTIFIED BY 'password123';
```

-- केवल SELECT अधिकार देना

```
GRANT SELECT ON Students TO student_user;
```

□ यहाँ DBA ने सुनिश्चित किया कि छात्र केवल डेटा देख सके, लेकिन उसे बदल न सके।

DBA के लाभ (Importance of DBA)

- डेटाबेस सुरक्षित और विश्वसनीय रहता है।
 - डेटा की सुसंगति और अखंडता बनी रहती है।
 - सिस्टम फेल होने पर भी डेटा सुरक्षित रहता है।
 - उपयोगकर्ताओं को उनकी आवश्यकता अनुसार एक्सेस मिलता है।
-

Topic 09 Database User

परिभाषा (Definition)

डेटाबेस यूज़र्स वे लोग होते हैं जो डेटाबेस सिस्टम का उपयोग करते हैं। हर उपयोगकर्ता की अलग-अलग भूमिका और जिम्मेदारी होती है। DBMS विभिन्न प्रकार के उपयोगकर्ताओं को उनकी आवश्यकता अनुसार इंटरफ़ेस और सुविधाएँ प्रदान करता है।

डेटाबेस यूज़र्स के प्रकार (Types of Database Users)

1. **डेटाबेस एडमिनिस्ट्रेटर (DBA)**
 - डेटाबेस का प्रबंधन और रखरखाव करता है।
 - सुरक्षा, बैकअप, रिकवरी और प्रदर्शन अनुकूलन की जिम्मेदारी।
 2. **एप्लिकेशन प्रोग्रामर (Application Programmer)**
 - डेटाबेस से जुड़ी एप्लिकेशन विकसित करता है।
 - SQL और प्रोग्रामिंग भाषाओं का उपयोग करके डेटा एक्सेस करता है।
 - उदाहरण: छात्र पोर्टल या लाइब्रेरी मैनेजमेंट सिस्टम बनाने वाले डेवलपर्स।
 3. **एंड यूज़र (End User)**
 - सीधे डेटाबेस का उपयोग करता है।
 - केवल वही डेटा देखता है जो उसके लिए आवश्यक है।
 - उदाहरण: छात्र अपने मार्क्स देखता है, शिक्षक सभी छात्रों का परिणाम देखता है।
 - एंड यूज़र्स के प्रकार:
 - **Casual Users:** कभी-कभी क्वेरी चलाते हैं।
 - **Naive Users:** प्री-डिफाईंड फॉर्मों का उपयोग करते हैं (जैसे ATM यूज़र)।
 - **Sophisticated Users:** जटिल क्वेरी और एनालिसिस करते हैं।
 - **Stand-alone Users:** अपने कंप्यूटर पर डेटाबेस एप्लिकेशन चलाते हैं।
 4. **सिस्टम एनालिस्ट (System Analyst)**
 - डेटाबेस की आवश्यकताओं का विश्लेषण करता है।
 - तय करता है कि डेटाबेस में कौन-कौन सी टेबल्स और रिलेशनशिप होंगी।
-

आरेख (Diagram)

[DBA] → Database Management

[Application Programmer] → Develop Applications

[End Users] → Use Applications

[System Analyst] → Analyze Requirements

उदाहरण (Example)

मान लीजिए एक **University Database** है:

- **DBA:** डेटाबेस की सुरक्षा और बैकअप सुनिश्चित करता है।
 - **Application Programmer:** छात्र पोर्टल बनाता है।
 - **End User (Student):** पोर्टल पर लॉगिन करके अपने मार्क्स देखता है।
 - **System Analyst:** तय करता है कि Students, Courses और Results टेबल्स कैसे जुड़ी होंगी।
-

लाभ (Importance of Database Users)

- डेटाबेस को विभिन्न दृष्टिकोणों से उपयोग किया जा सकता है।
 - हर उपयोगकर्ता अपनी भूमिका निभाकर सिस्टम को प्रभावी बनाता है।
 - डेटा सुरक्षित और संगठित रहता है।
-

UNIT SECOND

Topic 01 Entity Relationship Model

ER मॉडल डेटाबेस डिज़ाइन की एक तकनीक है जिसमें वास्तविक दुनिया की वस्तुओं (Entities) और उनके बीच के संबंधों (Relationships) को ग्राफिकल रूप में दर्शाया जाता है।

मुख्य घटक

घटक	विवरण
Entity	कोई वस्तु जैसे छात्र, शिक्षक, कोर्स
Attribute	Entity के गुण जैसे नाम, रोल नंबर
Key	यूनिक पहचानकर्ता जैसे RollNo
Relationship	दो या अधिक Entities के बीच संबंध

उदाहरण: छात्र और कोर्स का ER डायग्राम

[Student] ---enrolled_in---> [Course]

Attributes: Student (Name, RollNo), Course (CourseID, Title)

Topic 02 Mapping Constraints

परिभाषा

Mapping constraints यह दर्शाते हैं कि एक Entity कितनी बार दूसरी Entity से जुड़ सकती है।

प्रकार	उदाहरण
1:1	एक छात्र का एक ही लाइब्रेरी कार्ड
1:N	एक शिक्षक कई छात्रों को पढ़ा सकता है
M:N	कई छात्र कई कोर्स कर सकते हैं

विषय: Keys

Key	कार्य
Primary Key	यूनिक पहचान जैसे RollNo
Candidate Key	संभावित Primary Keys
Foreign Key	दूसरे टेबल से लिंक करने वाला कॉलम

Topic 03 Entity Relationship Diagram

ER Diagram डेटाबेस डिज़ाइन की एक ग्राफिकल तकनीक है। इसमें वास्तविक दुनिया की वस्तुओं (Entities), उनके गुण (Attributes), और उनके बीच संबंध (Relationships) को प्रतीकों के माध्यम से दर्शाया जाता है।

ER Diagram के प्रतीक (Symbols)

प्रतीक	नाम	विवरण
□ (Rectangle)	Entity	वास्तविक वस्तु जैसे Student, Course
○ (Ellipse)	Attribute	Entity का गुण जैसे Name, RollNo
○ (Ellipse, Underlined)	Primary Key	यूनिक पहचानकर्ता जैसे RollNo
◆ (Diamond)	Relationship	Entities के बीच संबंध जैसे Enrolled_in
□ (Double Ellipse)	Multivalued Attribute	एक से अधिक मान रखने वाला गुण जैसे PhoneNumbers
○ (Dashed Ellipse)	Derived Attribute	किसी अन्य Attribute से निकला हुआ मान जैसे Age (DOB से निकला)
▷ (Triangle)	Generalization	कई Entities को एक सामान्य Entity में जोड़ना
Rectangle around Diamond	Aggregation	Relationship को Entity की तरह ट्रीट करना
Lines with 1:1, 1:N, M:N	Mapping Constraints	संबंध की Cardinality दर्शाना

उदाहरण: Student और Course का ER Diagram

- Entities:
 - Student (RollNo, Name, Email)
 - Course (CourseID, Title)
- Relationship:

- Enrollment (Student और Course के बीच)
- **Constraints:**
 - एक Student कई Courses में नामांकित हो सकता है (1:N)
 - एक Course कई Students को पढ़ा सकता है (M:N)

SQL Representation

```
CREATE TABLE Student (  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(100)  
);
```

```
CREATE TABLE Course (  
    CourseID INT PRIMARY KEY,  
    Title VARCHAR(50)  
);
```

```
CREATE TABLE Enrollment (  
    RollNo INT,  
    CourseID INT,  
    FOREIGN KEY (RollNo) REFERENCES Student(RollNo),  
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```



E-R Diagram

Symbols



Object that requires data storage

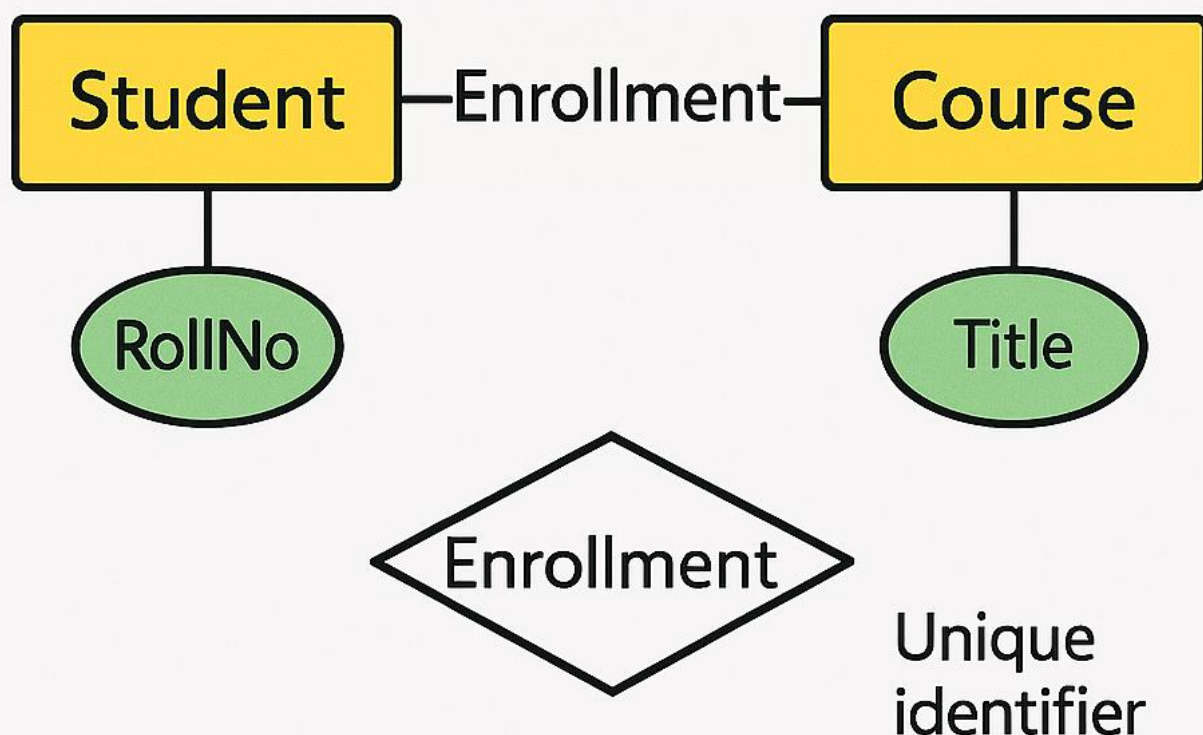


Property of an entity



Association between entities

■ Key



Topic 04 Generalization and Aggregation

आरेख में शामिल प्रतीक:

- **एंटिटी (Entity):** पीले रंग के आयत (rectangle) – जैसे छात्र, पाठ्यक्रम
- **एट्रिब्यूट (Attribute):** हरे रंग के अंडाकार (ellipse) – जैसे नाम, शीर्षक
- **प्राइमरी की (Primary Key):** अंडरलाइन किया गया एट्रिब्यूट – जैसे रोलानक
- **रिलेशनशिप (Relationship):** सफेद रंग का हीरा (diamond) – जैसे नामांकन
- **जनरलाइजेशन (Generalization):** त्रिकोण प्रतीक – कई एंटिटीज को एक सामान्य वर्ग में जोड़ता है
- **एग्रीगेशन (Aggregation):** रिलेशनशिप को आयत में घेरकर दर्शाना – जैसे एक प्रोजेक्ट में छात्र और गाइड दोनों शामिल हों
- **मैपिंग कंस्ट्रेंट्स (Mapping Constraints):** रेखाओं पर 1:1, 1:N, M:N जैसे लेबल – एंटिटीज के बीच संबंध की संख्या दर्शाते हैं

Generalization and Aggregation

- **Generalization:** Combining entities with common properties into a generalized entity.

Example: Teacher and Clerk fall under the 'Employee' entity.

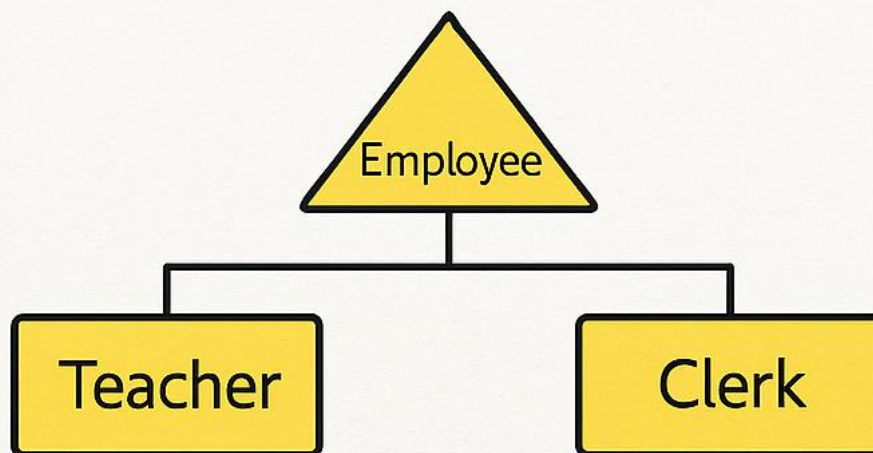
- **Aggregation**
Treating a relationship between entities as a higher-level entity.

Example: Student and Guide work on a 'Project.'

Generalization and Aggregation

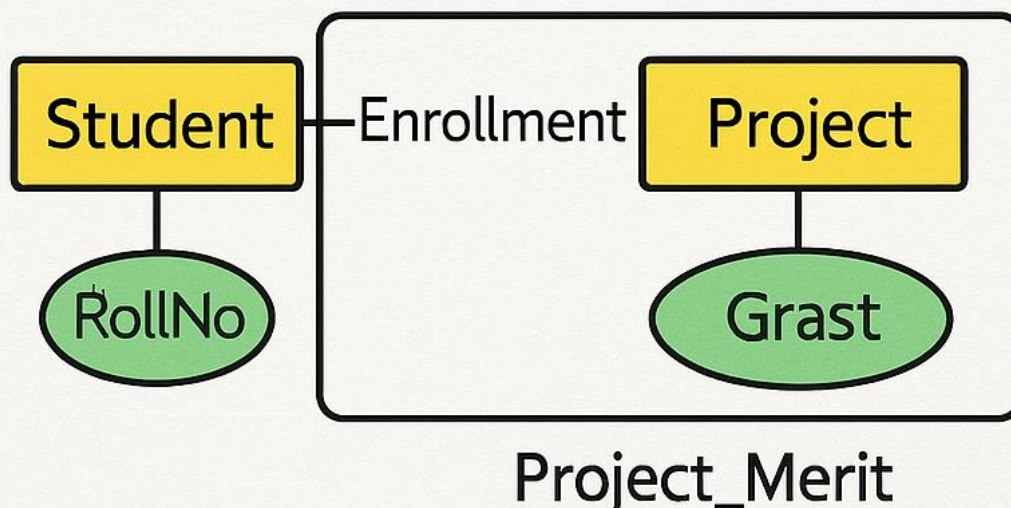
■ Generalization

Combining entities with common properties into a generalized entity.



■ Aggregation

Treating a relationship between entities as a higher-level entity



Topic 05 Normalization

डेटा को इस तरह व्यवस्थित करना कि दोहराव और विसंगति कम हो।

सामान्य रूपें (Normal Forms)

Normal Form	विशेषता
1NF	प्रत्येक कॉलम में Atomic Values
2NF	1NF + कोई Partial Dependency नहीं
3NF	2NF + कोई Transitive Dependency नहीं
BCNF	Candidate Key पर पूर्ण निर्भरता
4NF	कोई Multivalued Dependency नहीं
5NF	कोई Join Dependency नहीं

उदाहरण: Normalization

RollNo	Name	Course1	Course2
101	Amit	DBMS	OS

नॉर्मलाइज़ेशन के उदाहरण (Examples of Normal Forms)

1NF – First Normal Form

शर्त: प्रत्येक कॉलम में केवल Atomic (एकल) मान होना चाहिए।

Before 1NF:

RollNo	Name	Courses
101	Amit	DBMS, OS
102	Ravi	Networking

After 1NF:

RollNo	Name	Course
101	Amit	DBMS
101	Amit	OS
102	Ravi	Networking

अब प्रत्येक सेल में केवल एक मान है।

2NF – Second Normal Form

शर्त: 1NF + कोई Partial Dependency नहीं होनी चाहिए (Composite Key के हिस्से पर निर्भरता नहीं होनी चाहिए)।

Before 2NF:

RollNo	Course	StudentName
101	DBMS	Amit
101	OS	Amit

यहाँ StudentName केवल RollNo पर निर्भर है, न कि पूरे Composite Key (RollNo, Course) पर।

After 2NF:

Students Table:

RollNo	StudentName
101	Amit

Enrollment Table:

RollNo	Course
101	DBMS
101	OS

अब Partial Dependency को हटा दिया गया है।

3NF – Third Normal Form

शर्त: 2NF + कोई Transitive Dependency नहीं होनी चाहिए (एक non-key attribute दूसरे non-key attribute पर निर्भर नहीं होना चाहिए)।

Before 3NF:

RollNo	Name	DeptID	DeptName
101	Amit	D01	CS
102	Ravi	D02	IT

यहाँ DeptName → DeptID → RollNo है, यानी Transitive Dependency है।

After 3NF:

Students Table:

RollNo	Name	DeptID
101	Amit	D01
102	Ravi	D02

Department Table:

DeptID	DeptName
D01	CS
D02	IT

अब Transitive Dependency को हटा दिया गया है।

BCNF – Boyce-Codd Normal Form

शर्त: प्रत्येक Determinant एक Candidate Key होना चाहिए।

Before BCNF:

Teacher	Subject	Room
A	Math	101
B	Physics	102

मान लीजिए एक रूम में केवल एक विषय पढ़ाया जा सकता है $\rightarrow \text{Room} \rightarrow \text{Subject}$

लेकिन Room Candidate Key नहीं है।

After BCNF:

Room_Subject Table:

Room	Subject
101	Math
102	Physics

Teacher_Subject Table:

Teacher	Subject
A	Math
B	Physics

अब सभी Determinants Candidate Keys हैं।

4NF – Fourth Normal Form

शर्त: कोई Multivalued Dependency नहीं होनी चाहिए।

Before 4NF:

Student	Course	Hobby
Amit	DBMS	Cricket
Amit	OS	Cricket
Amit	DBMS	Music
Amit	OS	Music

यहाँ $\text{Student} \twoheadrightarrow \text{Course}$ और $\text{Student} \twoheadrightarrow \text{Hobby}$ दो स्वतंत्र Multivalued Dependencies हैं।

After 4NF:

Student_Course Table:

Student	Course
Amit	DBMS
Amit	OS

Student_Hobby Table:

Student	Hobby
Amit	Cricket
Amit	Music

अब Multivalued Dependencies को अलग कर दिया गया है।

5NF – Fifth Normal Form

शर्त: कोई Join Dependency नहीं होनी चाहिए।

Before 5NF:

Student	Course	Instructor
---------	--------	------------

Amit	DBMS	Prof. A
Amit	OS	Prof. B
Ravi	DBMS	Prof. A

यहाँ मान लीजिए कि Student और Course का Instructor से संबंध केवल Join से ही समझा जा सकता है।
After 5NF:

Student_Course Table:

Student	Course
Amit	DBMS
Amit	OS
Ravi	DBMS

Course_Instructor Table:

Course	Instructor
DBMS	Prof. A
OS	Prof. B



UNIT THIRD

Topic 01 Oracle RDBMS क्या है?

Oracle RDBMS (Relational Database Management System) एक शक्तिशाली डेटाबेस सॉफ्टवेयर है जो डेटा को रिलेशनल मॉडल में संगठित करता है। यह क्लाइंट-सर्वर आर्किटेक्चर पर आधारित होता है और बड़े पैमाने पर एप्लिकेशन के लिए उपयोग किया जाता है।

Oracle Architecture के मुख्य घटक

1. Kernel (कोर)

- Oracle का मुख्य भाग जो डेटा प्रोसेसिंग, स्टोरेज और सुरक्षा को नियंत्रित करता है।
- इसमें SQL प्रोसेसर, डेटा मैनेजर, और ट्रांजैक्शन कंट्रोल शामिल होते हैं।

2. System Global Area (SGA)

- यह एक मेमोरी क्षेत्र है जो Oracle Instance के लिए आरक्षित होता है।
- इसमें कैश, बफर और लॉग से संबंधित डेटा रहता है।

SGA के घटक: | घटक | कार्य | |-----|-----| | **Database Buffer Cache** | डेटा ब्लॉक्स को स्टोर करता है | | **Redo Log Buffer** | ट्रांजैक्शन लॉग को स्टोर करता है | | **Shared Pool** | SQL स्टेटमेंट्स और डेटा डिक्शनरी को स्टोर करता है | | **Large Pool / Java Pool** | विशेष कार्यों के लिए अतिरिक्त मेमोरी |

Topic 02 Background Processes (पृष्ठभूमि प्रक्रियाएँ)

Database Writer (DBWn)

- SGA के Database Buffer Cache से बदले हुए डेटा को डिस्क पर लिखता है।

Log Writer (LGWR)

- Redo Log Buffer से डेटा को Online Redo Log Files में लिखता है।
- यह ट्रांजैक्शन रिकवरी के लिए आवश्यक होता है।

Process Monitor (PMON)

- फेल हुए यूज़र प्रोसेस को साफ करता है।
- संसाधनों को मुक्त करता है और रिकवरी करता है।

System Monitor (SMON)

- सिस्टम क्रैश के बाद रिकवरी करता है।
- फ्री स्पेस को मैनेज करता है।

Checkpoint (CKPT)

- डेटा फाइल हेडर को अपडेट करता है जब चेकपॉइंट होता है।

Listener Registration (LREG)

- डेटाबेस इंस्टेंस की जानकारी को Listener से रजिस्टर करता है।

Topic 03 Database Files

प्रकार	विवरण
Data Files	वास्तविक डेटा को संग्रहित करते हैं

Redo Log Files	ट्रांजैक्शन लॉग को संग्रहित करते हैं
Control Files	डेटाबेस की संरचना और स्थिति की जानकारी रखते हैं

UNIT FOURTH

Topic 01 SQL क्या है?

SQL (Structured Query Language) एक मानक भाषा है जिसका उपयोग डेटाबेस से डेटा को परिभाषित करने, जोड़ने, संशोधित करने, हटाने और प्राप्त करने के लिए किया जाता है।

SQL डेटा प्रकार (SQL Data Types)

डेटा प्रकार	विवरण	उदाहरण
INT	पूर्णांक	RollNo INT
VARCHAR (n)	चर लंबाई वाला टेक्स्ट	Name VARCHAR(50)
DATE	दिनांक	DOB DATE
FLOAT	दशमलव संख्या	Marks FLOAT
BOOLEAN	सत्य/असत्य	IsActive BOOLEAN

Topic 02 SQL कमांड्स के प्रकार

1. DDL – Data Definition Language

डेटाबेस की संरचना को परिभाषित करने के लिए।

कमांड	कार्य	उदाहरण
CREATE	नई टेबल बनाना	

```
CREATE TABLE Students (
    RollNo INT PRIMARY KEY,
    Name VARCHAR(50),
    Course VARCHAR(50)
);
```

| ALTER | टेबल में बदलाव करना |

```
ALTER TABLE Students ADD Email VARCHAR(100);
```

| DROP | टेबल हटाना |

```
DROP TABLE Students;
```

| TRUNCATE | सभी डेटा हटाना (संरचना बनी रहती है) |

```
TRUNCATE TABLE Students;
```

2. DML – Data Manipulation Language

डेटा को जोड़ने, बदलने और हटाने के लिए।

कमांड	कार्य	उदाहरण
INSERT	डेटा जोड़ना	

```
INSERT INTO Students VALUES (101, 'Amit', 'DBMS');
```

| UPDATE | डेटा बदलना |

```
UPDATE Students SET Course = 'OS' WHERE RollNo = 101;
```

| DELETE | डेटा हटाना |

```
DELETE FROM Students WHERE RollNo = 101;
```

3. DQL – Data Query Language

डेटा प्राप्त करने के लिए।

कमांड	कार्य	उदाहरण
SELECT	डेटा देखना	

```
SELECT Name, Course FROM Students WHERE RollNo = 101;
```

4. TCL – Transaction Control Language

ट्रांजैक्शन को नियंत्रित करने के लिए।

कमांड	कार्य	उदाहरण
COMMIT	बदलाव स्थायी करना	

```
COMMIT;
```

| ROLLBACK | बदलाव वापस लेना |

```
ROLLBACK;
```

| SAVEPOINT | ट्रांजैक्शन में पॉइंट सेट करना |

```
SAVEPOINT Save1;
```

5. DCL – Data Control Language

डेटाबेस में एक्सेस और अनुमति नियंत्रित करने के लिए।

कमांड	कार्य	उदाहरण
GRANT	अधिकार देना	

```
GRANT SELECT ON Students TO user1;
```

| REVOKE | अधिकार हटाना |

```
REVOKE SELECT ON Students FROM user1;
```

Topic 03 Joins

Equi-Join

- जब दो टेबल्स को समान कॉलम (Equality Condition) के आधार पर जोड़ा जाता है।

```
SELECT Students.Name, Courses.Title
```

```
FROM Students, Courses
```

```
WHERE Students.CourseID = Courses.CourseID;
```

Non-Equi Join

- जब टेबल्स को किसी अन्य शर्त (जैसे <, >, BETWEEN) के आधार पर जोड़ा जाता है।

```
SELECT Employees.Name, SalaryGrades.Grade
```

```
FROM Employees, SalaryGrades
```

WHERE Employees.Salary BETWEEN SalaryGrades.MinSalary AND SalaryGrades.MaxSalary;

Self Join

- जब एक ही टेबल को खुद से जोड़ा जाता है।

```
SELECT E1.Name AS Employee, E2.Name AS Manager
FROM Employees E1, Employees E2
WHERE E1.ManagerID = E2.EmpID;
```

Other Joins

- Natural Join:** समान कॉलम नामों पर आधारित।
- Cross Join:** Cartesian Product देता है।

```
SELECT * FROM Students CROSS JOIN Courses;
```

Aggregate Functions

Function	कार्य	उदाहरण
COUNT()	पंक्तियों की संख्या	SELECT COUNT(*) FROM Students;
SUM()	कुल योग	SELECT SUM(Marks) FROM Students;
AVG()	औसत	SELECT AVG(Marks) FROM Students;
MIN()	न्यूनतम मान	SELECT MIN(Marks) FROM Students;
MAX()	अधिकतम मान	SELECT MAX(Marks) FROM Students;

Math Functions

Function	कार्य	उदाहरण
ABS()	Absolute Value	SELECT ABS(-10); → 10
ROUND()	राउंड ऑफ करना	SELECT ROUND(12.345, 2); → 12.35
CEIL()	ऊपर की ओर पूर्णांक	SELECT CEIL(12.3); → 13
FLOOR()	नीचे की ओर पूर्णांक	SELECT FLOOR(12.9); → 12
MOD()	शेषफल	SELECT MOD(10, 3); → 1

String Functions

Function	कार्य	उदाहरण
LENGTH()	स्ट्रिंग की लंबाई	SELECT LENGTH('Oracle'); → 6
UPPER()	बड़े अक्षरों में बदलना	SELECT UPPER('sql'); → SQL
LOWER()	छोटे अक्षरों में बदलना	SELECT LOWER('SQL'); → sql
SUBSTR()	उप-स्ट्रिंग निकालना	SELECT SUBSTR('Database', 1, 4); → Data
CONCAT()	स्ट्रिंग जोड़ना	SELECT CONCAT('Data', 'Base'); → Database

GROUP BY Clause

- GROUP BY** का उपयोग डेटा को समूहित करने के लिए किया जाता है।

```
SELECT Course, AVG(Marks)
FROM Students
GROUP BY Course;
```


UNIT FIFTH

Topic 01 PL/SQL (Procedural Language/SQL)

परिभाषा

PL/SQL Oracle द्वारा विकसित एक Procedural Extension है जो SQL को प्रोग्रामिंग सुविधाओं (जैसे लूप्स, कंडीशन्स, वेरिएबल्स) के साथ जोड़ता है।

यह SQL को अधिक शक्तिशाली और लचीला बनाता है।

Basics of PL/SQL

- **Block Structure:** PL/SQL प्रोग्राम ब्लॉक्स में लिखा जाता है।
- **Block के भाग:**
 1. **DECLARE** – वेरिएबल्स और कर्सर घोषित करना
 2. **BEGIN** – मुख्य लॉजिक और SQL स्टेटमेंट्स
 3. **EXCEPTION** – त्रुटि हैंडलिंग
 4. **END** – ब्लॉक का अंत

उदाहरण:

```
DECLARE
    v_name VARCHAR2(50);
BEGIN
    SELECT Name INTO v_name FROM Students WHERE RollNo = 101;
    DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_name);
END;
```

Topic 02 Data Types in PL/SQL

Data Type	विवरण	उदाहरण
NUMBER	संख्यात्मक मान	salary NUMBER(8,2);
VARCHAR2	टेक्स्ट	name VARCHAR2(50);
DATE	दिनांक	dob DATE;
BOOLEAN	TRUE/FALSE	isActive BOOLEAN;

Control Structures

1. IF-THEN-ELSE

```
IF marks >= 40 THEN
    DBMS_OUTPUT.PUT_LINE('Pass');
ELSE
    DBMS_OUTPUT.PUT_LINE('Fail');
END IF;
```

2. LOOP

```
FOR i IN 1..5 LOOP
```

```
DBMS_OUTPUT.PUT_LINE('Number: ' || i);  
END LOOP;
```

Database Access with PL/SQL

- SQL स्टेटमेंट्स को PL/SQL ब्लॉक में एम्बेड किया जा सकता है।

```
BEGIN  
    UPDATE Students SET Marks = Marks + 5 WHERE Course = 'DBMS';  
END;
```

Database Connections

- PL/SQL प्रोग्राम Oracle Database से जुड़कर SQL स्टेटमेंट्स को निष्पादित करता है।
- यह क्लाईंट-सर्वर आर्किटेक्चर पर आधारित है।

Transaction Management

- **COMMIT:** बदलाव स्थायी करना
- **ROLLBACK:** बदलाव वापस लेना
- **SAVEPOINT:** ट्रांजैक्शन में पॉइंट सेट करना

```
BEGIN  
    UPDATE Students SET Marks = 90 WHERE RollNo = 101;  
    COMMIT;  
END;
```

Database Locking

- जब एक ट्रांजैक्शन डेटा को बदल रहा होता है, तो उस डेटा पर लॉक लगाया जाता है।
- इससे Concurrency Control सुनिश्चित होता है।

Cursor Management

- **Cursor** का उपयोग क्वेरी से प्राप्त कई पंक्तियों को एक-एक करके प्रोसेस करने के लिए किया जाता है।

उदाहरण:

```
DECLARE  
    CURSOR c_students IS SELECT Name FROM Students;  
    v_name Students.Name%TYPE;  
BEGIN  
    OPEN c_students;  
    LOOP  
        FETCH c_students INTO v_name;  
        EXIT WHEN c_students%NOTFOUND;  
        DBMS_OUTPUT.PUT_LINE('Student: ' || v_name);  
    END LOOP;  
    CLOSE c_students;  
END;
```